Feasibility Report: AI-Driven Natural Language Filtering

Title: AI-Driven Natural Language Filtering for Silver Bucket
Date: 16-May-2025
Prepared by: Preetham Saldhana,Nithin Nayak
Reviewed by: Prakhyath Rai, Suraj Nagraj
Version: v1.0

Contents

Section No.	Section Title	Page Number
1.	Executive Summary	3
2.	Overview	3
3.	Objective	3
4.	Scope	3
5.	Theory	4
6.	Workflow Description	8
7.	Feasibility Analysis	9
8.	Technical Feasibility	10
9.	Operational Feasibility	11
10.	Economic Feasibility	12
11.	Legal and Regulatory Compliance	12
12.	Recommendations	13
13.	Risks and Mitigations	13
14.	Result and Analysis	13
15.	Appendices	13
16.	Limitations & Conclusion	14

1. Executive Summary

This application is designed to enhance user experience by allowing users to filter and access extensive data (employees, allocations, availability, projects, competencies, etc.) using natural language queries. This enhancement will streamline data navigation, making it more intuitive for users such as resource managers, project managers, senior management, and employees.

2. Overview

This document proposes the development of backend solution based on Fastapi which will allow users to filter and access extensive data using natural language queries.

3. Objective

The objective of this document is to outline the functionality, benefits, and feasibility of integrating an AI-driven NLP feature, powered by a large language model (LLM), into the Silverbucket resource allocation platform. It aims to provide a comprehensive overview of how this enhancement will work, its potential impact on user productivity, and the technical requirements for its implementation.

4. Scope

- **Input:** User sends a query string via REST API.
- **Processing:** The backend sends the query to an LLM endpoint (e.g., AWS Bedrock) which parses it and extracts filters.
- **Output:** A structured JSON object containing fields like project name, user role, date range, etc.

Boundaries

- LLM capabilities are limited to text parsing; it will not perform business logic.
- The backend will not enforce UI-level constraints or render outputs visually.
- Multi-tenancy and access control must be enforced post-filter extraction.

5. Theory

1. FastAPI

FastAPI is a modern, asynchronous web framework for building APIs with Python 3.7+ using standard type hints. It is built on top of Starlette (for web handling) and Pydantic (for data validation). It supports automatic generation of OpenAPI documentation and JSON Schema.

Advantages

- High performance, comparable to Node.js and Go.
- Automatic request validation and serialization.
- Built-in support for OAuth2, JWT, and CORS.
- Interactive API documentation (Swagger UI, ReDoc).
- Asynchronous support for non-blocking I/O.

Limitations

- Requires understanding of asynchronous programming.
- Smaller ecosystem compared to Flask or Django.
- No built-in admin interface.

Use Cases

- RESTful APIs and microservices.
- Backend for web and mobile applications.
- Machine learning model deployment.
- Real-time applications using WebSockets.

2. AWS Bedrock

AWS Bedrock is a fully managed service that provides access to foundation models (FMs) from multiple providers (e.g., Anthropic, AI21 Labs, Cohere,

Meta, Stability AI, Amazon) via API. It allows developers to build and scale generative AI applications without managing infrastructure.

Advantages

- Serverless and fully managed.
- Access to multiple foundation models.
- Integration with AWS services (S3, Lambda, SageMaker).
- Secure and compliant with enterprise standards.

Limitations

- Higher cost for large-scale usage.
- Limited customization compared to self-hosted models.
- Tied to AWS ecosystem.

Use Cases

- Generative AI applications (text, image, code).
- Chatbots and virtual assistants.
- Enterprise AI solutions.
- Content summarization and translation.

3. PostgreSQL

PostgreSQL is an open-source, object-relational database system known for its reliability, extensibility, and standards compliance. It supports advanced features like ACID transactions, MVCC, custom data types, and full-text search.

Advantages

- Strong consistency and ACID compliance.
- Advanced indexing and query optimization.

- JSONB support for semi-structured data.
- Extensible with custom functions and extensions.

Limitations

- Steeper learning curve than some alternatives.
- Performance tuning can be complex.
- Not ideal for high-throughput, low-latency workloads.

Use Cases

- Web and enterprise applications.
- Geospatial applications (with PostGIS).
- Data warehousing and analytics.
- Financial and transactional systems.

4. Amazon ECR (Elastic Container Registry)

Amazon ECR is a fully managed Docker container registry that allows developers to store, manage, and deploy container images. It integrates with ECS, EKS, and AWS Lambda, and supports private and public repositories.

Advantages

- Seamless integration with AWS services.
- Secure image storage with IAM policies.
- High availability and scalability.
- Automated image vulnerability scanning.

Limitations

- Vendor lock-in with AWS.
- Costs can increase with large image storage.

- Fewer community features compared to Docker Hub.

Use Cases

- Storing and managing Docker images.
- CI/CD pipelines for containerized applications.
- Versioning and deploying microservices.

5. Amazon ECS (Elastic Container Service)

Amazon ECS is a fully managed container orchestration service that supports Docker containers. It allows users to run and scale containerized applications using EC2 or Fargate launch types. ECS manages scheduling, scaling, and networking.

Advantages

- Deep integration with AWS ecosystem.
- Fargate mode eliminates server management.
- Supports blue/green deployments.
- Cost-effective for predictable workloads.

Limitations

- Less flexible than Kubernetes (EKS).
- Complex networking for large-scale apps.
- Vendor lock-in.

Use Cases

- Microservices architecture.
- Web applications and APIs.
- Batch processing and background jobs.
- Event-driven applications with Lambda and SQS.

6. Workflow Description

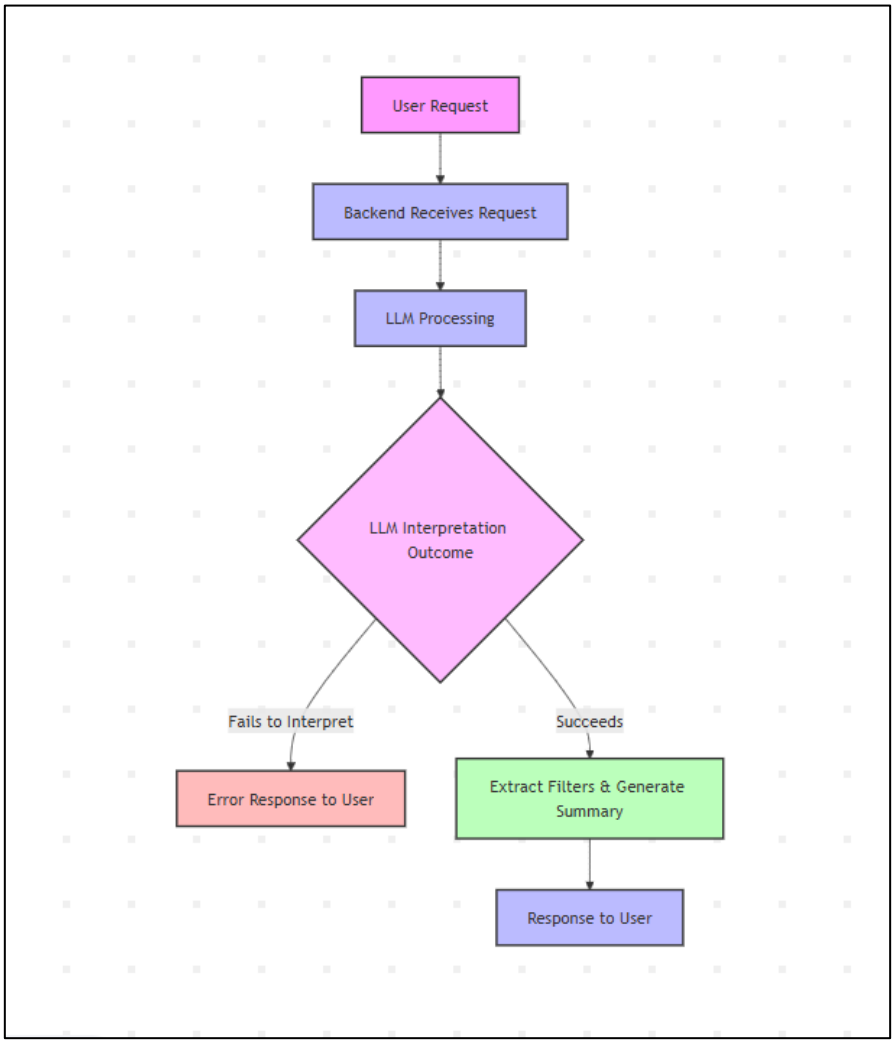

Figure 6 a: Flow Diagram

1. **User Request**

   The user sends a request containing a natural language query and parameters filters of which value must be extracted.

2. **Backend Receives Request**

   The backend receives and validates the request payload.

3. **LLM Processing**

   The backend sends the query to the LLM for interpretation.

4. **LLM Interpretation Outcome**

- If the LLM **fails** to interpret the query, an error response is returned to the user.
- If the LLM **succeeds**, it extracts the filters and generates a summary.

5. **Response to User**

   The backend sends the extracted filters and the interpreted query summary back to the user.

The flow of the application is as depicted in the figure 6 a and 6b .

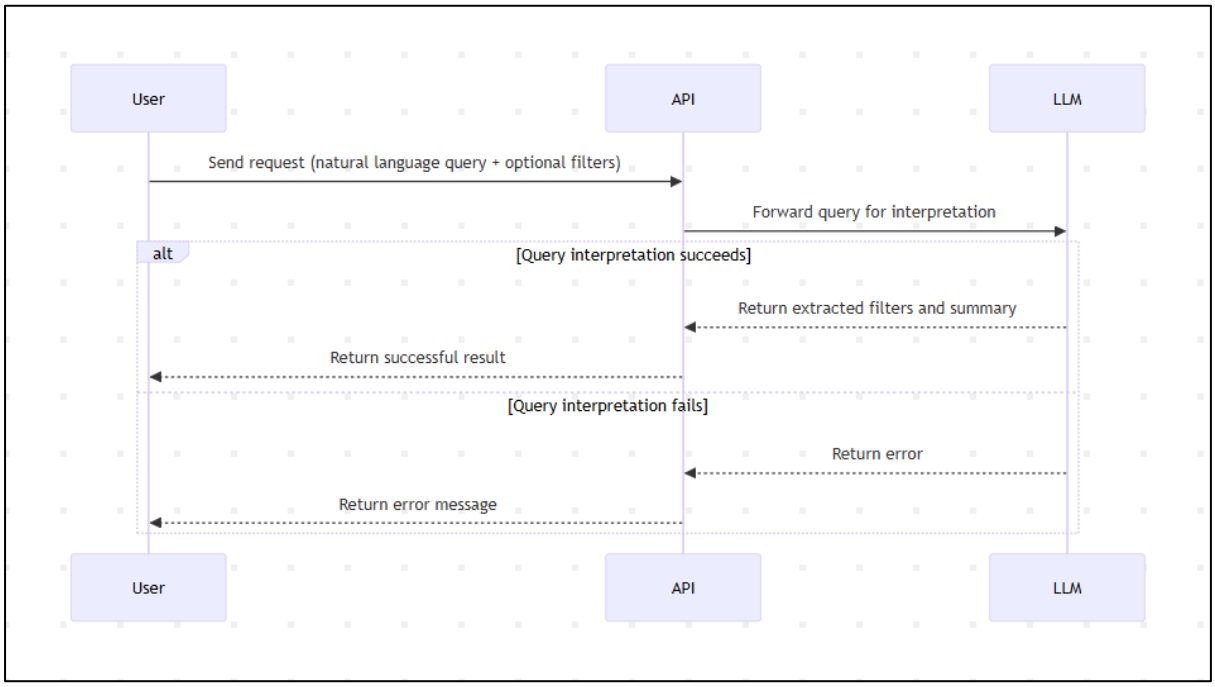

Figure 6 b: Sequence Diagram

7. Feasibility Analysis

a. Technical Feasibility

Fast API is a lightweight, scalable, and efficient framework for backend services. Integrating a large language model (LLM) is straightforward, utilizing APIs or local deployments to ensure seamless functionality.

b. Operational Feasibility

The backend can be integrated into existing services, pipelines, or systems with minimal overhead, ensuring smooth and efficient operation within the current infrastructure.

c. Economic Feasibility

The implementation is cost-effective, particularly when leveraging open-source components or usage-based LLM APIs, which can significantly reduce overall expenses.

8. Technical Feasibility

8.1 Technologies Used

Component	Technology
API Backend	FastAPI (Python)
LLM Integration	AWS Bedrock
Data Storage	PostgreSQL
Security	JWT

8.2 Technical Workflow

1. **Input:** User sends a POST request to /parse-query with a natural language string.
2. **Processing:**
    - Backend sends the prompt to AWS Bedrock.
    - LLM returns structured filters (e.g., {"project": "X", "role": "Developer", "date_range": "Q1"}).
    - Backend performs post-validation and access scoping.

3. **Output:** Structured JSON response returned to client for further filtering.

8.3 Integration

Service	Status
AWS Bedrock	Compatible
PostgreSQL	Existing
Role-based filters	Enforceable
Multi-tenancy	Requires isolation at post-processing level

8.4 Performance Considerations

- Latency: LLM response may take 300–1000ms.
- Caching common queries could reduce load.
- Async support in FastAPI ensures efficient handling of concurrent requests.

9. Operational Feasibility

- No frontend required — API-first design.
- Role-based filters will be enforced after parsing to ensure tenant and access integrity.
- Logs and tracing available via AWS CloudWatch.

10. Economic Feasibility

10.1 Capex

Component	Cost Per Month ($)
Elastic Load Balancing	22.5
EC2 Postgres	9.648
EC2 Jumpbox	9.648
Route 53	0.5
Domain registration	14
Data transfer	20
ECR (Container Registry)	0.1
VPC	74.9
Total	151.296

10.2 Opex

Based on the resource utilization of ECS and AWS Batch services, along with token processing assumptions, the estimated operational expenditure (OPEX) per filter execution is approximately $0.0218, accounting for compute, memory, and token handling costs.

10.2 Benefits

- Reduced query effort for end users.
- Improved accessibility for non-technical teams.
- Faster turnaround on resource planning tasks.

11. Legal and Regulatory Compliance

- Data never stored or processed beyond prompt scope.
- All API requests authenticated and auditable.

12. Recommendations

- Proceed with pilot implementation in a staging environment.
- Validate filter accuracy with domain-specific test cases.
- Evaluate real-time performance under production loads.

13. Risks and Mitigations

Risk	Mitigation
LLM misinterprets vague input	Implement prompt templates and validation
Slow LLM response time	Async processing or caching of common queries
API misuse or overload	Rate limiting and authentication required

14. Results & Analysis

- Initial tests show ~90% accuracy in extracting filters with properly engineered prompts.
- Integration with AWS Bedrock successful via REST API.

15. Appendices

15.1 Dependencies

- Python 3.11+
- FastAPI
- AWS SDK (Boto3 / HTTP client)
- PostgreSQL
- JWT Auth handler

15.2 Screenshots

The figure 15.2 :a depicts the natural language query and filter params to be extracted from the user query. Whereas the figure 15.2: b depicts the filters extracted from the natural language query.

```
{
  "query": {
    "user_query": "Show all engineers available next week with AWS skills"
  },
  "params": [
    "role","cloud"
  ]
}
```

Figure 15.2 a: Input Payload

```
{
  "status": 200,
  "data": {
    "role": "engineers",
    "cloud": "AWS",
    "user_query": "Show all engineers available next week with AWS skills",
    "summary": {
      "interpretation_details": "Extracted 'engineers' as the role and 'AWS' as the cloud skill.",
      "assumptions": "The query doesn't specify values for other parameters that might be in the full
list of params."
    }
  }
}
```

Download

Figure 15.2 b: Output Payload

16. Limitations

- LLM output may vary depending on prompt clarity.
- Dynamic data like dropdowns are not yet linked to real-time filters.
- LLMs may occasionally hallucinate or omit fields unless prompts are precise.

17. Conclusion

The proposed FastAPI-based backend is highly feasible for implementation and scalable for various use cases. Leveraging LLMs for natural language query parsing reduces the complexity of data access and supports broader integration possibilities. The system architecture is modular, maintainable, and future proof for enhancements like schema awareness or fine-tuned models.